

A1 + A2

1. Find employees who are either in departments with more than 5 employees or have a job title with a minimum salary above 10000, or both. Exclude those in departments where the manager earns less than their department's average.

```
-- Employees in departments with more than 5 employees
SELECT e.employee_id, e.first_name, e.last_name, 'Large Dept'
AS type
FROM HR.EMPLOYEES e
WHERE e.department_id IN (
    SELECT department_id
    FROM HR.EMPLOYEES
    GROUP BY department_id
    HAVING COUNT(*) > 5
)

UNION

-- Employees with a job title with minimum salary above 10000
SELECT e.employee_id, e.first_name, e.last_name, 'High Min Job'
AS type
FROM HR.EMPLOYEES e
JOIN HR.JOBS j ON e.job_id = j.job_id
WHERE j.min_salary > 10000

MINUS

-- Exclude employees in departments where the manager earns
less than department average
SELECT e.employee_id, e.first_name, e.last_name, '' AS type
FROM HR.EMPLOYEES e
JOIN HR.DEPARTMENTS d ON e.department_id = d.department_id
JOIN HR.EMPLOYEES m ON d.manager_id = m.employee_id
WHERE m.salary < (
    SELECT AVG(salary)
    FROM HR.EMPLOYEES
    WHERE department_id = m.department_id
);
```

2. For each country, count the number of departments. Display only the country_name and department_count, in ascending order of the country_name. Include the countries having no departments, too.

```

SELECT
    c.country_name,
    COUNT(d.department_id) AS department_count
FROM
    hr.countries c
JOIN
    hr.locations l ON c.country_id = l.country_id
LEFT OUTER JOIN
    hr.departments d ON l.location_id = d.location_id
GROUP BY
    c.country_name
ORDER BY
    c.country_name ASC;

```

3. For each department, find the employee_id, full_name, salary, department_name, and job title of the second-highest-paid employee(s) i.e. employee(s) having the second-highest salary. If a department has fewer than two employees, do not include it in the results. Display the output in descending order of the salary. If two employees have the same salary, prioritize the one whose department name is lexicographically smaller. If a tie still exists, prioritize the employee with the lower employee_id.

```

SELECT
    e1.employee_id,
    e1.first_name || ' ' || e1.last_name AS full_name,
    e1.salary,
    d1.department_name,
    j1.job_title
FROM
    hr.employees e1
JOIN
    hr.departments d1 ON e1.department_id = d1.department_id
JOIN
    hr.jobs j1 ON e1.job_id = j1.job_id
WHERE
    1 = (
        SELECT COUNT(DISTINCT e2.salary)
        FROM hr.employees e2
        WHERE e2.department_id = e1.department_id
        AND e2.salary > e1.salary
    )
    AND 2 <= (
        SELECT COUNT(*)
        FROM hr.employees e3
        WHERE e3.department_id = e1.department_id
    )
ORDER BY
    e1.salary DESC,

```

```
d1.department_name ASC,  
e1.employee_id ASC;
```

4. Find the employee_id, first_name, and salary of employees in descending order of the salary and ascending order of the employee ID, who meet **exactly one** of the following two conditions:
- They report to a manager whose salary is greater than 15000.
 - They work in a department located in 'Seattle'.

```
SELECT * FROM (  
  -- (A UNION B) MINUS (A INTERSECT B)  
  (  
    SELECT e.employee_id, e.first_name, e.salary  
    FROM hr.employees e  
    JOIN hr.employees m ON e.manager_id = m.employee_id  
    WHERE m.salary > 15000  
  
    UNION  
  
    SELECT e.employee_id, e.first_name, e.salary  
    FROM hr.employees e  
    JOIN hr.departments d ON e.department_id =  
d.department_id  
    JOIN hr.locations l ON d.location_id = l.location_id  
    WHERE l.city = 'Seattle'  
  
    MINUS  
  
    SELECT e.employee_id, e.first_name, e.salary  
    FROM hr.employees e  
    JOIN hr.employees m ON e.manager_id = m.employee_id  
    JOIN hr.departments d ON e.department_id =  
d.department_id  
    JOIN hr.locations l ON d.location_id = l.location_id  
    WHERE m.salary > 15000 AND l.city = 'Seattle'  
  )  
)  
ORDER BY salary DESC, employee_id ASC;
```

5. Find employees (first and last name), their departments, and salary, for those who earn more than the average salary in their own department. Only consider departments where there is at least one employee earning less than the company average salary and at least one earning more than the company average salary. Use a CASE statement to categorize salary as 'High' (if above 10,000), 'Medium' (if between 5,000 and 10,000), or 'Low' (if below 5,000).

```
SELECT
```

```

    e.first_name,
    e.last_name,
    d.department_name,
    e.salary,
    CASE
        WHEN e.salary > 10000 THEN 'High'
        WHEN e.salary BETWEEN 5000 AND 10000 THEN 'Medium'
        ELSE 'Low'
    END AS salary_category
FROM
    HR.employees e
JOIN
    HR.departments d ON e.department_id = d.department_id
WHERE
    e.salary > (
        SELECT AVG(e2.salary)
        FROM HR.employees e2
        WHERE e2.department_id = e.department_id
    )
    AND e.department_id IN (
        SELECT e3.department_id
        FROM HR.employees e3
        GROUP BY e3.department_id
        HAVING
            SUM(CASE WHEN e3.salary < (SELECT AVG(e4.salary)
FROM HR.employees e4) THEN 1 ELSE 0 END) > 0
            AND SUM(CASE WHEN e3.salary > (SELECT
AVG(e4.salary) FROM HR.employees e4) THEN 1 ELSE 0 END) > 0
        );

```

B1 + B2

1. Find all employees who earn more than the salary of at least five distinct employees who work in a different department but are located in the same city as the first employee's department. Display the employee_id, full_name, department_name, and salary of these employees, in descending order of their department_name. If the department name is the same, break the tie by choosing the employee with more salary and lower employee_id.

```
SELECT
    e1.employee_id,
    e1.first_name || ' ' || e1.last_name AS full_name,
    d1.department_name,
    e1.salary
FROM
    hr.employees e1
JOIN
    hr.departments d1 ON e1.department_id = d1.department_id
JOIN
    hr.locations l1 ON d1.location_id = l1.location_id
WHERE
    (
        SELECT COUNT(DISTINCT e2.employee_id)
        FROM hr.employees e2
        JOIN hr.departments d2 ON e2.department_id =
d2.department_id
        JOIN hr.locations l2 ON d2.location_id = l2.location_id
        WHERE
            d2.department_id != d1.department_id
            AND l2.city = l1.city
            AND e1.salary > e2.salary
    ) >= 5
ORDER BY
    d1.department_name DESC,
    e1.salary DESC,
    e1.employee_id;
```

2. Find the departments located in Americas that have a department manager whose salary is between 10000 and 15000 (inclusive), and manages at least two employees. Do not include the departments which has no manager. Show only the department's name, the manager's full name, salary of the manager, in descending order of the manager's salary and ascending order of the department name.

```
SELECT
    d.department_name,
    m.first_name || ' ' || m.last_name AS manager_name,
    m.salary AS manager_salary
FROM
```

```

        hr.departments d
    JOIN
        hr.locations l ON d.location_id = l.location_id
    JOIN
        hr.countries c ON l.country_id = c.country_id
    JOIN
        hr.regions r ON c.region_id = r.region_id
    JOIN
        hr.employees m ON d.manager_id = m.employee_id
    WHERE
        r.region_name = 'Americas'
        AND m.salary BETWEEN 10000 AND 15000
        AND (
            SELECT COUNT(*)
            FROM hr.employees e
            WHERE e.manager_id = d.manager_id
        ) >= 2
    ORDER BY
        m.salary DESC,
        d.department_name ASC;

```

3. List departments where at least one employee has held every job title currently assigned to employees in that department (based on job history). For each such department, display the department name and a CASE statement indicating whether the department's average salary is 'Above' or 'Below or equal' to the company average.

```

SELECT
    d.department_name,
    CASE
        WHEN AVG(e.salary) > (SELECT AVG(salary) FROM
HR.employees) THEN 'Above Company Average'
        ELSE 'Below or Equal to Company Average'
    END AS salary_status
FROM
    HR.departments d
JOIN
    HR.employees e ON d.department_id = e.department_id
WHERE
    EXISTS (
        SELECT 1
        FROM HR.employees e2
        WHERE e2.department_id = d.department_id
        AND NOT EXISTS (
            SELECT 1
            FROM (
                SELECT DISTINCT e3.job_id

```

```

        FROM HR.employees e3
        WHERE e3.department_id = d.department_id
    ) dept_jobs
    WHERE NOT EXISTS (
        SELECT 1
        FROM HR.job_history jh
        WHERE jh.employee_id = e2.employee_id
        AND jh.job_id = dept_jobs.job_id
    )
    )
)
GROUP BY
    d.department_id, d.department_name;

```

Alternative Solution:

```

SELECT
    d.department_name,
    CASE
        WHEN (SELECT AVG(e_sub.salary) FROM HR.employees e_sub
WHERE e_sub.department_id = d.department_id) >
            (SELECT AVG(salary) FROM HR.employees)
        THEN 'Above Company Average'
        ELSE 'Below or Equal to Company Average'
    END AS salary_status
FROM
    HR.departments d
WHERE
    -- Check if at least one employee in this department has
    held ALL job titles currently in this department
    EXISTS (
        SELECT 1
        FROM HR.employees e_main
        WHERE e_main.department_id = d.department_id
        AND (
            -- Count distinct job titles from job_history for
            this specific employee
            SELECT COUNT(DISTINCT jh.job_id)
            FROM HR.job_history jh
            WHERE jh.employee_id = e_main.employee_id
            AND jh.job_id IN ( -- Ensure job_id is one of the
            current department's jobs
                SELECT DISTINCT e_current.job_id
                FROM HR.employees e_current
                WHERE e_current.department_id = d.department_id
            )
        ) = (
            -- Count total distinct job titles currently

```

```

assigned in this department
        SELECT COUNT(DISTINCT e_current.job_id)
        FROM HR.employees e_current
        WHERE e_current.department_id = d.department_id
    )
);

```

4. Find employees who are either managers of more than one department (by being the manager of employees from multiple departments) or who have a salary greater than the average salary of all employees, or both. Exclude those who work in departments located in 'Seattle'. Additionally, print a type for each employee as either “Multi-Dept Manager” or “Above Avg Salary” (for employees satisfying both the criteria, printing either of the types will suffice).

```

-- Employees who manage people from more than one department
SELECT DISTINCT m.employee_id, m.first_name, m.last_name,
'Multi-Dept Manager' AS type
FROM HR.EMPLOYEES m
JOIN HR.EMPLOYEES e ON m.employee_id = e.manager_id
GROUP BY m.employee_id, m.first_name, m.last_name
HAVING COUNT(DISTINCT e.department_id) > 1

UNION

-- Employees who have a salary greater than the average salary
of all employees
SELECT employee_id, first_name, last_name, 'Above Avg Salary'
AS type
FROM HR.EMPLOYEES
WHERE salary > (SELECT AVG(salary) FROM HR.EMPLOYEES)

MINUS

-- Exclude employees whose department is in 'Seattle'
SELECT e.employee_id, e.first_name, e.last_name, '' AS type
FROM HR.EMPLOYEES e
JOIN HR.DEPARTMENTS d ON e.department_id = d.department_id
JOIN HR.LOCATIONS l ON d.location_id = l.location_id
WHERE l.city = 'Seattle';

```

5. For each department, show the department_name, employee_id, and salary of the highest-paid employee(s). If a department has no employees, do not include it in the results. Display the output in descending order of the salary. If two employees have the same salary, prioritize the one whose department name is lexicographically larger. If there still exists a tie, prioritize the employee with

the lower employee_id.

```
SELECT
    d.department_name,
    e.employee_id,
    e.salary
FROM
    hr.employees e
JOIN
    hr.departments d ON e.department_id = d.department_id
WHERE
    e.salary = (
        SELECT MAX(e2.salary)
        FROM hr.employees e2
        WHERE e2.department_id = e.department_id
    )
ORDER BY
    e.salary DESC,
    d.department_name DESC,
    e.employee_id ASC;
```

C1 + C2

1. List departments where **every employee** has at least one job change and display the department name and the number of employees. Use a CASE statement to show if the department's average salary is 'Above' or 'Below or equal to' the company average.

```
SELECT
    d.department_name,
    COUNT(e.employee_id) AS num_employees,
    CASE
        WHEN AVG(e.salary) > (SELECT AVG(salary) FROM
HR.employees) THEN 'Above Company Average'
        ELSE 'Below or Equal to Company Average'
    END AS salary_status
FROM
    HR.departments d
JOIN
    HR.employees e ON d.department_id = e.department_id
WHERE
    NOT EXISTS (
        SELECT 1
        FROM HR.employees e2
        WHERE e2.department_id = d.department_id
        AND NOT EXISTS (
            SELECT 1
            FROM HR.job_history jh
            WHERE jh.employee_id = e2.employee_id
        )
    )
GROUP BY
    d.department_id, d.department_name;
```

2. Find employees who are either among the top 3 highest-earning employees in their department or have a job title not held by anyone else in their department, or both. Exclude those who work in a department with a manager whose first name is 'Steven'. Additionally, print a type for each employee as either "Top 3 Salary" or "Unique Job" (for employees satisfying both the criteria, printing either of the types will suffice).

```
-- Employees in top 3 salaries per department
SELECT e.employee_id, e.first_name, e.last_name, 'Top 3 Salary'
AS type
FROM HR.EMPLOYEES e
WHERE (
```

```

        SELECT COUNT(*)
        FROM HR.EMPLOYEES e2
        WHERE e2.department_id = e.department_id AND e2.salary >
e.salary
    ) < 3

UNION

-- Employees with a job title not held by anyone else in their
department
SELECT e.employee_id, e.first_name, e.last_name, 'Unique Job'
AS type
FROM HR.EMPLOYEES e
WHERE NOT EXISTS (
    SELECT 1
    FROM HR.EMPLOYEES e2
    WHERE e2.department_id = e.department_id
    AND e2.job_id = e.job_id
    AND e2.employee_id <> e.employee_id
)

MINUS

-- Exclude employees who have ever worked in a department with
manager named 'Steven'
SELECT e.employee_id, e.first_name, e.last_name, '' AS type
FROM HR.EMPLOYEES e
JOIN HR.DEPARTMENTS d ON e.department_id = d.department_id
JOIN HR.EMPLOYEES m ON d.manager_id = m.employee_id
WHERE m.first_name = 'Steven';

```

3. List the department_name, city, and country_name for departments that have no employees hired before July 1, 2000, have a manager, and the manager's job is not titled 'Stock Manager'. Display only departments that have at least one employee in ascending order of country and descending order of city. If there still exists a tie, display the lexicographically smaller department_name first.

```

SELECT
    d.department_name,
    l.city,
    c.country_name
FROM
    hr.departments d
JOIN
    hr.locations l ON d.location_id = l.location_id

```

```

JOIN
    hr.countries c ON l.country_id = c.country_id
-- Ensure department has a manager
JOIN
    hr.employees m ON d.manager_id = m.employee_id
JOIN
    hr.jobs j ON m.job_id = j.job_id
WHERE
    j.job_title != 'Stock Manager'
    -- Department has at least one employee
    AND EXISTS (
        SELECT 1
        FROM hr.employees e
        WHERE e.department_id = d.department_id
    )
    AND NOT EXISTS (
        SELECT 1
        FROM hr.employees e
        WHERE e.department_id = d.department_id
            AND e.hire_date < TO_DATE('2000-07-01', 'YYYY-MM-DD')
    )
ORDER BY
    c.country_name ASC,
    l.city DESC,
    d.department_name ASC;

```

4. Find all employees who have a job title with a minimum salary greater than the average minimum salary of all jobs, and work in departments that have more than 5 employees.

```

SELECT e.employee_id, e.first_name, e.last_name, e.job_id,
j.min_salary, d.department_name
FROM HR.EMPLOYEES e
JOIN HR.JOBS j ON e.job_id = j.job_id
JOIN HR.DEPARTMENTS d ON e.department_id = d.department_id
WHERE j.min_salary > (
    SELECT AVG(min_salary)
    FROM HR.JOBS
)
AND e.department_id IN (
    SELECT department_id
    FROM HR.EMPLOYEES
    GROUP BY department_id
    HAVING COUNT(*) > 5
)
ORDER BY j.min_salary DESC;

```

5. For each department, calculate its total salary expense. Categorize this expense as 'Low' (total salary $\leq$ 15000), 'Medium' (total salary $>$ 15000 and $\leq$ 50000), 'High' (total salary $>$ 50000 and $\leq$ 100000), or 'Very High' (total salary $>$ 100000). Display department_name, total_salary_expense, and expense_category in descending order of the total salary expense and ascending order of department_name. Include departments with no employees (their total expense should be 0 and categorized as 'Low').

```
SELECT
    d.department_name,
    NVL(SUM(e.salary), 0) AS total_salary_expense,
    CASE
        WHEN NVL(SUM(e.salary), 0) <= 15000 THEN 'Low'
        WHEN NVL(SUM(e.salary), 0) <= 50000 THEN 'Medium'
        WHEN NVL(SUM(e.salary), 0) <= 100000 THEN 'High'
        ELSE 'Very High'
    END AS expense_category
FROM
    hr.departments d
LEFT JOIN
    hr.employees e ON d.department_id = e.department_id
GROUP BY
    d.department_name
ORDER BY
    total_salary_expense DESC,
    d.department_name ASC;
```